Build sequence and design review

A dependency-driven plan for the one-brace experiment, independent package comparison, and one-page decision sheet.

Recommendation: build the first release in React and TypeScript, with SVG rendering and a pure TypeScript structural solver. Keep the solver behind a small interface so it can be replaced later. This document plans implementation; it does not authorize or include application code.

Why TypeScript is sufficient for this scope

The specified bridge has 10 nodes and two displacement variables per node. Its three support restraints leave a **17 by 17 system**. One ranking evaluates five designs; all three load positions require only 15 small solves. This size is derived from the supplied specification, not from a performance benchmark. [1]

TypeScript becomes JavaScript at runtime. JavaScript Number and Rust f64 both use IEEE 754 double precision; selecting Rust does not by itself increase numerical precision. Algorithm choice, conditioning, units, and verification determine whether the results deserve trust. [3-5]

Option	Assessment for Forma
TypeScript throughout	Recommended. One language, shared contracts, direct browser execution, and simplest integration. Verify actual latency on the demo laptop.
TypeScript UI + Rust/Wasm solver	Viable future boundary. Adds a second build toolchain, data conversion, binary loading, and integration tests. Use when profiling or a proven existing Rust solver justifies it. [6]
C++/Wasm or a Python service	No demonstrated advantage for this fixed fixture. C++ adds a similar native integration boundary; a Python service adds another runtime and deployment process.

Performance decision rule

Proposed acceptance targets: after warm-up, the full five-design calculation has p95 time below 8 ms and a snapped load change paints its result within 50 ms on the presentation laptop. These are targets to measure, not promised results. If missed, profile computation and rendering separately. Recompute only when the snapped joint changes; then consider versioned caching or a worker before a language rewrite. A worker can move work off the main thread. [7]

Scope baseline: 10 N downward load; B1, B2, or B3; one added brace P1-P4; fixed geometry and E/A; session-only trials; locally served production build; browser print. Costs remain a separately labeled scenario. [1,2]

DELIVERY MAP

Dependencies and merge order

Numbers identify handoffs. They are not a requirement to finish all of Phase 2 before beginning Phase 3 or Phase 4.

Prerequisite convention: every listed prerequisite must be merged and its acceptance checks passing on the shared integration branch. Work can be drafted earlier against agreed fixtures, but cannot be considered integrated until those gates pass.

Wave	Eligible subphases	Outcome
A	1.1 -> 1.2 -> 1.3	Scope, contracts, and runnable foundation
B	2.1 | 3.1 | 4.1	Structures, studio, and costs can progress independently
C	2.2 -> 2.3; 3.2; 4.2	Verified analysis, mock-driven interaction, cost interface
D	3.3 after 2.3 + 3.2; 4.3 after 3.3 + 4.2	Live prediction/retest and financial isolation
E	5.1 -> 5.2 -> 5.3	Frozen report data and verified print output
F	6.1 -> 6.2 -> 6.3	Integrated regression, offline release, rehearsal
G	Optional: 7.1 and 7.2 after their listed gates	Measurements and assembly template

Assign one writer to each boundary

Owner	Exclusive implementation boundary
I - Integration	App shell, reducer/history, shared contracts and fixtures, src/community/report, shared configuration and dependency lockfile.
S - Structures	src/structures and structural tests; proposes fixture corrections to I with independent evidence.
U - Studio	src/studio and its component tests; consumes reducer state and callbacks, never owns a second trial store.
C - Community	src/community/costs, src/community/measurements, src/community/template and their tests.

These are proposed repository boundaries. The original documents name Vaibhav for integration; confirm the remaining people when assigning work. A role can own multiple subphases, but two people should not edit its shared files concurrently. [2]

Small pull requests that are easy to merge

Use one branch per subphase, for example phase/2.2-truss-solver. Each PR states its prerequisite PRs, owned files, acceptance evidence, and assumptions. Only I changes shared types or dependencies, in a preceding PR. Keep mocks at test or development boundaries; never silently substitute them for a failed live solve. Before merging, update from the integration branch and run the relevant tests, type check, and production build.

PHASE 1 / FOUNDATION

Fix the scope and contracts

Owner I establishes the interfaces that allow the other owners to work independently.

1.1 Record the first-release decisions

Owner: I, with all owners **Prerequisites:** None.

Owns: Planning notes and acceptance checklist.

Deliver: Adopt the technical specification for exact numerical behavior and the build plan for delivery priorities. Fix the load at 10 N and record mm, N, N/mm2, USD, year-end upkeep, and decimal real discount rate. Define required versus optional work.

Merge gate: All owners agree on three load positions, four independent one-brace alternatives, hidden rankings before commit, session-only history, separate cost scenarios, and a selected-trial report. No freeform bridge editing in this release.

1.2 Publish contracts and state ownership

Owner: I **Prerequisites:** 1.1.

Owns: Shared types, event definitions, error shapes, and module interface declarations.

Deliver: Define Design, Load, Analysis or AnalysisError, RankedBraces, Trial, QuoteInputs, Reading, and ReportPayload. Trial captures deep-copied results plus preset and solver versions. Report captures selected costs and readings. Separate active preview, retest, and saved trials. Define missing amount independently of zero.

Merge gate: S, U, and C can describe their inputs and outputs without importing another module's implementation. Commit is idempotent for the current round. Errors cannot masquerade as zero deflection. No global mutable store shared by physics and costs.

1.3 Create the runnable shell and fixtures

Owner: I; S reviews fixture data **Prerequisites:** 1.2.

Owns: App entry point, routes/tabs, shared fixture files, package setup, and test configuration.

Deliver: Set up the local React/TypeScript build, module folders, a test runner, and a production serve command. Store the supplied triangle, bridge displacement matrix, quote examples, and one immutable mock trial as independent expected data. Bundle assets locally.

Merge gate: The empty app builds and runs locally. All owners import the same contracts. Fixture values are copied from the specification with source labels; they are not generated by the new solver. Modules compile with explicit placeholders and no fabricated production results.

Phase exit

Merge 1.3 before opening the independent implementation lanes. Begin 2.1, 3.1, and 4.1 from the same foundation commit. This is the highest-value point for resolving naming or ownership disputes.

Prove the numerical engine

Owner S delivers a deterministic calculation module with no UI or financial dependencies.

2.1 Generate and validate the prescribed bridge

Owner: S **Prerequisites:** 1.3.

Owns: src/structures preset, geometry validation, and geometry tests.

Deliver: Generate B0-B4 and T0-T4 at 60 mm spacing and 40 mm height. Create 17 baseline members, three support restraints, and the four legal opposite diagonals. Each candidate adds one member to a fresh baseline. Validate endpoints, duplicates, lengths, E/A, loads, and finite numeric values.

Merge gate: Baseline has 10 nodes and 17 members; every candidate has 18. Every addition is sqrt(5200) mm long. Crossing diagonals do not create a node. Invalid models return typed errors with affected IDs.

2.2 Solve displacement and recover forces

Owner: S **Prerequisites:** 2.1.

Owns: src/structures stiffness assembly, linear solve, recovery, and solver tests.

Deliver: Assemble the direct stiffness matrix; constrain supports; solve the free system with pivoting or rank detection; recover reactions and axial forces. Tension is positive. Use scale-aware numerical tolerances and reject nonfinite or singular results; do not explicitly invert the matrix.

Merge gate: Triangle nonzero results agree within 1%. The unbraced square is unstable. Bridge baseline results meet the supplied 0.000001 mm tolerance. Check free-DOF residuals, reaction equilibrium, mirror symmetry, and sign conventions. Document chosen numerical tolerances.

2.3 Rank independent alternatives and expose failures

Owner: S **Prerequisites:** 2.2.

Owns: src/structures ranking, result adapter, and regression/performance checks.

Deliver: Run baseline plus P1-P4 under the same load and compare loaded-joint downward displacement. Compute gains without display rounding. Include all candidates within max(1e-9 mm, 1e-6 times best displacement). If baseline displacement is zero, return absolute difference only.

Merge gate: All 15 fixture displacements pass. B1 selects P1; B2 ties P2/P3; B3 selects P4. Baseline never accumulates braces. Errors return no current ranking. Record five-design timing on the demo laptop; no performance claim before measurement.

Phase exit

The solver is ready for the live adapter in 3.3. The fixture suite must remain valid if a later implementation uses Rust/Wasm. Solver tests and expected fixtures must not share the same calculation code.

PHASE 3 / STUDIO

Make the experiment understandable

U owns presentation and controls. I owns the reducer and saved trial state.

3.1 Draw the bridge and accessible controls

Owner: U **Prerequisites:** 1.3.

Owns: src/studio geometry view, panel selector, load controls, and view tests.

Deliver: Render the shared preset as SVG with support symbols, stable joint/panel labels, selected-brace preview, and snapped load positions. Offer native controls or arrow keys for B1-B3 and P1-P4. Keep rankings hidden.

Merge gate: Geometry agrees with the fixture. Mouse and keyboard select the same state. Pointer motion only changes the load when the snapped joint changes. Labels remain readable and selection does not depend only on color.

3.2 Implement commit and trial history against fixtures

Owner: I for reducer/history; U for view bindings **Prerequisites:** 3.1.

Owns: I: app state and reducer tests. U: prediction, history, and retest components.

Deliver: Model new round, pre-commit preview, commit/reveal, post-commit retest, saved-trial selection, and another prediction round. Use clearly isolated development fixtures until the live engine is connected. Preserve first-trial data independently from current controls.

Merge gate: Repeated commit produces one trial. Dragging after commit retains the chosen brace. New rounds hide rankings. Selecting a saved trial restores its own load, choice, and stored results without recomputing over them. Mock-only status is explicit during development.

3.3 Connect the live solver and comparison overlays

Owner: I for adapter; U for result presentation **Prerequisites:** 2.3 and 3.2.

Owns: I: live analysis adapter. U: overlays, force labels, result announcement, and error state.

Deliver: Replace development responses with the validated solver. Show original and modified results at the same load, all tied winners, and a separately labeled first trial. Apply one visible deformation scale to both overlays; announce committed results accessibly.

Merge gate: Commit P2 at B1: about 1.09% gain. Retest at B2: about 10.46% and a P2/P3 tie. Reopen the B1 trial unchanged. On active-analysis failure, clear active performance output while preserving valid history. No mock fallback remains in the production path.

Phase exit

A tester can complete the central experiment without explanation from a developer. The result language describes predicted loaded-joint deflection, not failure load or a universal strongest design.

PHASE 4 / COMMUNITY

Compare costs without coupling models

Owner C provides an independently testable financial calculation and interface.

4.1 Implement validated package calculations

Owner: C **Prerequisites:** 1.3.

Owns: src/community/costs calculations, validation, and unit tests.

Deliver: Capture material, freight, installation, credited cash contribution, annual upkeep, and each amount's source/status. Calculate upfront cost, funding gap, annuity factor, present cost, and break-even annual savings. Require nonnegative finite amounts/rate, positive integer years, and credits no larger than initial costs.

Merge gate: At 10 years and 3%, sample totals are $26,677.18 and $24,838.59; threshold is $234.46/year. At 0%, threshold is $200/year. Verify exact break-even with unrounded inputs, equal premiums, unknown versus zero, and partial outputs when upkeep is unknown.

4.2 Build editable packages and explain the result

Owner: C **Prerequisites:** 4.1.

Owns: src/community/costs form, summary view, and component tests.

Deliver: Provide equal-scope package labels, budget, horizon, and rate inputs. Show upfront gap separately from discounted total. Identify the higher-upfront package, expected annual savings, and the break-even threshold. Label every demo assumption and show unknown states.

Merge gate: Editing a quote or horizon updates the appropriate numbers. Raising only budget from $20,000 to $21,000 closes B's gap without changing present cost or threshold. A blank upkeep value leaves known upfront affordability visible.

4.3 Integrate navigation and verify domain isolation

Owner: I for app wiring; C for cost boundaries **Prerequisites:** 3.3 and 4.2.

Owns: I: navigation and integration checks. C: cost export adapter.

Deliver: Wire Compare build options into the same app while keeping quote state separate from the structural preset and trial history. Supply a serializable cost input/result snapshot for reporting. Record donated material at actual purchase cost with a note; avoid counting it twice.

Merge gate: Changing any quote, budget, or horizon leaves saved and current structural outputs identical. Changing load position leaves financial inputs and results identical. Missing cost data never prevents a valid structural report. Both scenarios have explicit labels.

Phase exit

The cost comparison is useful on its own and honest about its connection to the experiment. It does not imply the tabletop model certifies either community package. [1,2]

PHASE 5 / EVIDENCE

Print the intended trial reliably

I owns reporting. The report is an explicit snapshot, not a screenshot of whatever happens to be open.

5.1 Freeze the report payload

Owner: I **Prerequisites:** 3.3 and 4.3.

Owns: src/community/report payload builder, shared report types, and snapshot tests.

Deliver: Build an immutable payload from the selected saved trial, separately labeled optional retest, current selected quotes, and readings. Include trial ID, preset/solver versions, units, sources, and assumptions. Define the empty measurement state even if measurement entry is deferred.

Merge gate: Create a payload, then change active load, quote values, and selected trial. The payload remains unchanged. Reporting a saved trial uses its stored analysis, not a fresh solve. Unknown costs and absent measurements remain explicit.

5.2 Create a one-page print view

Owner: I **Prerequisites:** 5.1.

Owns: src/community/report print component, print styles, and report-only SVG adapter.

Deliver: Lay out US Letter with what changed, model prediction, physical measurement, cost comparison, and open assumptions. Include original/revised drawings, gains, winners, relevant signed forces, costs, and source labels. Reuse geometry rendering where possible.

Merge gate: Browser print preview is one page with readable type, units, and grayscale force labels. Both drawings use a stated common magnification. Long package names and unknown values do not overflow. Controls and navigation are excluded from printing.

5.3 Verify selected-trial export end to end

Owner: I, with U and C checking their outputs **Prerequisites:** 5.2.

Owns: Report integration tests and a saved sample PDF for release QA.

Deliver: Exercise export while a different retest is visible. Test complete quotes, missing upkeep, no measurements, and the longest supported labels. Verify numerical values and the selected scenario against the frozen payload.

Merge gate: Commit P2 at B1, retest at B2, then export the original trial. The main result remains B1/P2/about 1.09%; B2 is shown only if deliberately included and separately labeled. Every variant is legible on one page.

Phase exit

This phase ships Forma's one-page user decision sheet. The longer PDF you are reading is the team's implementation plan; it is not the application's report template.

PHASE 6 / RELEASE

Prove the complete demo

I owns the release gate. All owners resolve issues in their own boundaries.

6.1 Run integrated correctness and usability checks

Owner: I; S, U, and C review their modules **Prerequisites:** 2.3, 3.3, 4.3, and 5.3.

Owns: Cross-module tests and release acceptance record.

Deliver: Run numerical fixtures, input validation, hidden/revealed state transitions, retained trials, financial isolation, and selected-trial reporting. Have a new tester complete both views. Measure the latency targets on the presentation laptop.

Merge gate: No failing required acceptance test, visible stale result, mislabeled unit, or inaccessible essential control. Record measured performance and limitations. If slow, profile before choosing an optimization; correctness is not traded for speed.

6.2 Verify the offline production build

Owner: I **Prerequisites:** 6.1; also rerun after any optional feature merges.

Owns: Production artifact, local launch instructions, and release checklist.

Deliver: Install/build while online, then serve the built assets locally, disconnect internet, and reload. Run the whole flow using only the keyboard. Print the selected trial and inspect it. Ensure fonts, scripts, and any future Wasm file are local.

Merge gate: A clean start on the presentation laptop works without remote requests. Reload clears session trials as specified; the release test creates a new trial after reload. The local server remains running. One-page print and all required interactions pass.

6.3 Freeze and rehearse

Owner: I with all presenters **Prerequisites:** 6.2.

Owns: Tagged or otherwise fixed release commit, demo script, and submission materials.

Deliver: Run the 90-second sequence twice: predict, reveal/retest, compare costs, export, identify one assumption. Keep an offline copy and a known launch command. Freeze features; accept only correctness or demo-blocking fixes, then rerun affected release checks.

Merge gate: Two complete timed rehearsals succeed on the actual laptop. The delivered build is the tested commit. Required submission information and any reused-code disclosure are ready. Optional work cannot hold this gate open.

Schedule policy

The supplied plan targets integration at 1:30 PM, outside testing and print at 2:30 PM, laptop validation at 3:30 PM, feature freeze at 4:15 PM, and rehearsals by 4:40 PM, before the 5:00 PM ET deadline on November 7. These are source-plan targets, not estimates that this work has already met. Rebase timing to the actual start and staffing; preserve the dependency gates. [2]

PHASE 7 / OPTIONAL EXTENSIONS

Add evidence after the core works

These tasks are intentionally outside the required release dependency chain. Re-run the release checks if they are merged.

7.1 Record physical measurements with context

Owner: C; S reviews interpretation **Prerequisites:** 4.3 and 5.1; merge only after 6.1.

Owns: src/community/measurements and its report adapter.

Deliver: Capture design/version, applied load and position, measured joint, displacement, units, method, instrument resolution, and repeated raw readings. Associate readings with the correct case. Report inconclusive observations when the change cannot be resolved.

Merge gate: A reading for another load or design cannot appear as validation of the selected trial. Empty state remains valid. Readings are frozen in the report. A repeatable measurement with sufficient resolution is needed before claiming agreement with the model.

7.2 Export a separate assembly template

Owner: C; U reviews drawing reuse **Prerequisites:** 2.1 and 5.2; merge only after 6.1.

Owns: src/community/template and its print-only styles.

Deliver: Generate one side-panel pattern from the same geometry in millimeters. Print separately at 100% scale with a check bar. Mark labeled nodes, member lengths, and unconnected diagonal crossings clearly.

Merge gate: Measure the printed check bar on the actual printer. Template does not change or extend the one-page decision sheet. No fit-to-page scaling is accepted without verification. Optional template failure does not block the core release.

Cut order when time is short

Drop model-generated narration first, assembly template second, and live physical measurement third, as the build plan proposes. Keep prediction/commit/reveal, same-brace retest, cost comparison, and the selected-trial decision sheet. A speculative Rust port is also outside the first-release scope. [2]

Numerical anchors for review

Load	Original displacement	Best addition	Expected gain
B1	0.043702 mm	P1: 0.032189 mm	About 26.34%
B2	0.069936 mm	P2/P3: 0.062624 mm	About 10.46%
B3	0.043702 mm	P4: 0.032189 mm	About 26.34%

These are rounded expected fixtures from the specification, not newly measured or independently verified results. Implement the full 15-value regression matrix, not only this abbreviated review table. The predicted baseline-to-best change is about 0.0115 mm at B1 and 0.0073 mm at B2, which explains why physical measurement may be difficult. [1]

Questions to challenge the engine

The strongest technical objection is confidence in the model and its implementation, rather than raw execution speed.

Q1 What evidence would justify Rust now?

A small static solve does not establish a need for a native engine. Rust/Wasm is a reasonable future option, especially if the team already has a validated solver, but it introduces another integration boundary. [6]

Decision or test: Use the measured targets on page 1. Port only a demonstrated compute bottleneck or a compelling existing library. Keep analyze and rankBraces stable and rerun every numerical fixture after a port.

Q2 Does a faster language make the answer more accurate?

No automatic improvement follows from language choice. JavaScript Number and Rust f64 have the same standard precision class. Incorrect restraints, unstable matrices, or wrong signs can produce fast but misleading answers. [4,5]

Decision or test: Require pivoting/rank detection, scale-aware tolerances, residual/equilibrium checks, and the independent triangle fixture in 2.2. Never replace an unstable result with a plausible number.

Q3 Do the fixtures only prove that we copied the fixtures?

Tests can agree with an implementation while sharing its mistake. A visual match or a hardcoded lookup does not verify a general calculation.

Decision or test: Keep expected data independent. Use analytic triangle results, mirror symmetry, reaction balance, and load/EA scaling checks as additional evidence. Investigate discrepancies before changing expected values.

Q4 Is the physical bridge actually a pin-jointed truss?

The specified model treats bars as axial members and crossings as unconnected. Joint slip, bending, compression buckling, and out-of-plane motion are excluded. A string brace or a glued crossing changes those assumptions. [1]

Decision or test: Use model-qualified language. Have the structures owner document how a prototype differs, including whether its braces can carry compression. Do not imply a failure-load prediction.

Q5 Would a load-magnitude slider add meaningful exploration?

For this fixed linear model, scaling the same load scales displacements and forces; the percentage gains and rankings remain the same. This follows from the specified linear equations. Moving the load changes the comparison more meaningfully. [1]

Decision or test: Keep 10 N fixed initially. If magnitude is later exposed, label the linear assumption and test proportional response without implying behavior near failure.

Q6 Are we optimizing what the audience thinks we are?

Lowest displacement at one loaded joint is a narrow objective. It does not establish greatest strength, best buckling resistance, smallest displacement everywhere, or a universally best bridge. [1]

Decision or test: Use the label "best for loaded-joint deflection under this load." Show absolute movement, percentage change, and tied winners. Keep the same-brace retest central to the story.

Questions to challenge the demo

These questions identify ways the product could feel convincing while communicating more certainty than it has.

Q7 Can a ruler validate a hundredth of a millimeter?

The expected improvement is roughly 0.007-0.012 mm for the best placements in these fixtures. A reading whose resolution exceeds that change cannot establish the predicted difference. [1]

Decision or test: Record resolution and repeated readings. Label an unresolvable result inconclusive. If changing the prototype or measurement method, document and validate that new setup separately; keep the numerical fixture unchanged.

Q8 Why does a tabletop brace experiment lead to community costs?

The two views support different decisions. The supplied project quotes are fictional, and the structural fixture does not determine or certify those packages. Without an explicit explanation, the transition can look like an unsupported engineering recommendation. [1,2]

Decision or test: Introduce the second view as a separate equal-scope budgeting example. Label scenarios and sources. A future real linkage needs separately assessed designs, quantities, and supplier costs.

Q9 Are the packages really comparable over the chosen horizon?

A lower discounted cost can be misleading if scope, service life, replacement needs, or upkeep assumptions differ. The current formula assumes equal service over the horizon and constant real year-end upkeep. [1]

Decision or test: Capture a scope statement and source for each amount. State excluded replacement/residual costs. If equal service cannot be justified, flag the comparison rather than declaring a universal winner.

Q10 Could the report silently describe a different experiment?

Yes, if it reads current controls instead of a selected immutable trial. Cost edits and retesting make this easy to miss.

Decision or test: Freeze trial results, chosen costs, and readings in 5.1. Test export of B1 while B2 is visible. If the report includes a retest, label its own load and design separately.

Q11 Will users trust an exaggerated deformation drawing?

The actual movements are too small to see at the bridge drawing's scale. Different exaggeration factors can make one design appear more improved than it is.

Decision or test: Use the same labeled magnification for both overlays, retain the undeformed reference, and display numerical mm values. Test force labels and selections in grayscale and with a keyboard.

Q12 What happens after a reload or a network failure?

The chosen scope stores trials in memory and expects a local server. Offline capability does not promise persistence across reloads or a double-clickable file. [1]

Decision or test: Explain session-only history. Test disconnect-and-reload before starting a fresh trial. If durable history becomes a requirement, add a separate versioned persistence phase with migration and restore tests.

Traceability and open decisions

Requirements come from the supplied documents. File boundaries, merge waves, and latency targets are recommendations in this plan.

Decisions to record before implementation

Adopt in 1.1: TypeScript throughout; fixed 10 N load; three load positions; four candidates; one independent added brace; session-only history; no server-side calculations; independently labeled cost comparison. Confirm that the one-side model receives the specified 10 N; do not silently assume how a physical two-sided bridge shares a load.

Assign in 1.1: names for I, S, U, and C; the presentation laptop and browser; actual available build time; who reviews structural fixtures independently; whether optional physical evidence is feasible.

Measure before changing architecture: latency in 2.3/6.1, numerical residuals and regression agreement in 2.2/2.3, and measurement resolution in 7.1. No benchmark or physical validation has been performed as part of this planning document.

Project source documents

[1] **Forma Technical Specification.** Supplied Word document, dated 7 November 2026. File: Forma_Technical_Specification.docx. Basis for geometry, equations, contracts, fixtures, financial rules, and release criteria.

[2] **Forma NOVA Hackathon Build Plan.** Supplied Markdown document. File: Forma_NOVA_Build_Plan-2.md. Basis for team roles, judging sequence, checkpoint targets, and optional-feature cut order.

Primary technical references

[3] TypeScript Handbook - The Basics
Type annotations are removed; TypeScript preserves JavaScript runtime behavior.
https://www.typescriptlang.org/docs/handbook/2/basic-types.html

[4] MDN - JavaScript Number
JavaScript numeric representation and precision.
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Number

[5] Rust standard library - f64
Rust double-precision floating-point type.
https://doc.rust-lang.org/std/primitive.f64.html

[6] MDN - WebAssembly concepts
Compilation targets including Rust, and JavaScript/WebAssembly integration.
https://developer.mozilla.org/en-US/docs/WebAssembly/Guides/Concepts

[7] MDN - Using Web Workers
Background worker execution and communication with the main thread.
https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API/Using_web_workers

Technical references checked 7 November 2026. The language recommendation is an assessment of this specified workload, not a claim that TypeScript outperforms Rust generally.